11. Déboguer les erreurs des appels API

Lorsque vous appelez le backend depuis Flutter (souvent via une classe du modèle ou un service du type `UserApi`, `ReservationApi`, etc.), une exception peut survenir à plusieurs endroits : échec réseau, code HTTP inattendu, corps JSON invalide, ou encore erreur dans `fromJson` (champ manquant, mauvais type, etc.).

Dans ce dernier cas, le message d'erreur réel est souvent très parlant (par exemple un `type 'Null' is not a subtype of type 'String'` parce que `phone` est défini en `String` au lieu de `String?`). Pourtant, **rien n'apparaît dans la console de débogage** si l'appel est déclenché depuis le `build` d'un provider Riverpod : l'exception remonte jusqu'au provider, qui passe en état d'erreur, sans forcément afficher la stack trace complète.

Pour diagnostiquer ce genre de problème rapidement, **entourez le corps de vos méthodes API d'un `try` / `catch` qui affiche l'erreur et la *stack trace* en console** avec `debugPrint` avant de relancer une exception (ou de la transformer) comme auparavant.

Exemple

Avant :

lib/features/user/data/user_api.dart

```dart
static Future<User> getUser() async {
  final response = await ApiClient.get('get_user');
  if (response.statusCode == 200) {
    return User.fromJson(json.decode(response.body));
  } else {
    throw Exception('Failed to get user');
  }
}
```

Après :

lib/features/user/data/user_api.dart

```dart
import 'package:flutter/foundation.dart';

static Future<User> getUser() async {
  try {
    final response = await ApiClient.get('get_user');
    if (response.statusCode == 200) {
      return User.fromJson(json.decode(response.body));
    } else {
```

```
      throw Exception('Failed to get user');
    }
  } catch (e, st) {
    debugPrint('$e\n$st');
    throw Exception('Failed to get user');
  }
}
```

Quelques remarques :

- Utilisez `catch (e, st)` pour récupérer aussi la **stack trace** (`st`), indispensable pour localiser la ligne en cause.
- `debugPrint` envoie le message vers la console de débogage de Flutter / IntelliJ ; c'est l'équivalent recommandé d'un `print` en développement.

Complément : observateur des providers

Même avec ce `try` / `catch`, il reste utile d'activer un observateur global sur vos providers pour voir quand ils passent en erreur et avec quel message. Voir la FAQ 10. Outil de débogage des providers.

En combinant les deux (journalisation dans la couche API **et** observateur Riverpod), vous disposez en général de toutes les informations nécessaires pour corriger un modèle mal typé, un endpoint mal appelé ou une réponse JSON inattendue.